

UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ
CÂMPUS SANTA HELENA
BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO

Classificação de sentimentos em comentários com Naive Bayes no Weka

Trabalho Prático 1

Leonardo José Reis Pinto

Sistemas Inteligentes Aplicados
Prof. Thiago Naves

Santa Helena
2026

Sumário

1	Introdução	2
2	O algoritmo Naive Bayes	2
3	Base de dados	3
3.1	Origem	3
3.2	Consolidação e tratamento	4
3.3	A base no formato ARFF	4
3.4	Transformação do texto em atributos	6
4	Como os testes foram feitos	6
5	Resultados	8
5.1	Naive Bayes sobre a base, com 10 folds	8
5.2	Variando o número de folds	10
5.3	Ligando a discretização supervisionada	11
5.4	Comparação com outros classificadores	14
6	Resumo dos resultados	16
7	Conclusão	16
	Referências	17
A	Como reproduzir os resultados	17
B	Saídas do Weka com 5 e 20 folds	18

1 Introdução

Saber se um comentário fala bem ou fala mal de alguma coisa parece trivial para quem lê, mas deixa de ser quando existem milhares de comentários por dia. Uma loja que vende pela internet, um restaurante ou um serviço de streaming recebem esse volume o tempo todo, e ler tudo na mão não é viável. É nesse ponto que entra a classificação automática de sentimento, que atribui a cada texto um rótulo positivo ou negativo a partir do que o algoritmo aprendeu em exemplos anteriores.

Este trabalho monta esse experimento do começo ao fim. Parto de três bases públicas de comentários (IMDb, Amazon e Yelp), junto tudo em um único arquivo, trato as inconsistências, converto para o formato ARFF, que é o formato de trabalho do Weka, e avalio o classificador Naive Bayes sobre essa base. Os testes são feitos com validação cruzada em três configurações diferentes e também com a opção *useSupervisedDiscretization* ligada, como pede o enunciado. No fim comparo o Naive Bayes com outros classificadores do próprio Weka, para ter uma ideia de onde ele se encaixa nesse problema.

2 O algoritmo Naive Bayes

O Naive Bayes é um classificador probabilístico. Ele não procura uma fronteira que separa as classes no espaço, como faz uma SVM, nem constrói uma sequência de testes, como uma árvore de decisão. O que ele faz é calcular, para cada classe possível, qual a probabilidade daquela classe ter gerado o exemplo que está sendo analisado, e devolver a classe com a maior probabilidade.

A base de tudo é o teorema de Bayes. Para uma classe c e um exemplo descrito pelos atributos $x_1, x_2, \dots, x_n$, temos

$$P(c \mid x_1, \dots, x_n) = \frac{P(c) \cdot P(x_1, \dots, x_n \mid c)}{P(x_1, \dots, x_n)}. \quad (1)$$

O denominador é igual para todas as classes, então pode ser ignorado na hora de comparar. O problema está em $P(x_1, \dots, x_n \mid c)$: estimar a probabilidade conjunta de todos os atributos exigiria uma quantidade de dados que quase nunca existe. A saída do algoritmo é supor que, dada a classe, os atributos são independentes entre si. Com essa suposição, a probabilidade conjunta vira um produto:

$$P(c \mid x_1, \dots, x_n) \propto P(c) \prod_{i=1}^n P(x_i \mid c). \quad (2)$$

A classe escolhida é aquela que maximiza esse valor. A suposição de independência é o motivo do nome *naive*, ingênuo em inglês, porque ela quase nunca é verdadeira. Em um texto, a palavra “not” e a palavra “good” claramente não aparecem de forma independente. O que se observa na prática, e isso já foi bastante estudado na literatura,

é que mesmo com a suposição errada o algoritmo costuma acertar a classe, porque para decidir basta que a ordem entre as probabilidades esteja correta, não os valores absolutos.

A forma de estimar $P(x_i | c)$ depende do tipo do atributo. Para atributos nominais, o Weka usa a contagem de ocorrências de cada valor dentro de cada classe, com uma correção para evitar probabilidade zero. Para atributos numéricos, a implementação padrão assume que os valores seguem uma distribuição normal dentro de cada classe e estima média e desvio padrão a partir dos dados de treino. Essa diferença é exatamente o que a opção *useSupervisedDiscretization* altera, e por isso ela aparece na Seção 5.3.

As vantagens do Naive Bayes são o treino muito rápido, já que basta percorrer a base uma vez contando frequências, o baixo consumo de memória e o fato de ele funcionar bem mesmo com poucos exemplos. A limitação principal é a própria suposição de independência, que prejudica o resultado quando os atributos são muito correlacionados, e o fato de as probabilidades que ele devolve serem pouco calibradas, geralmente próximas de zero ou de um.

3 Base de dados

3.1 Origem

A base usada é a *Sentiment Labelled Sentences*, do repositório de aprendizado de máquina da UCI. Ela foi montada por Kotzias et al. para um artigo do KDD 2015 e reúne frases retiradas de avaliações de três sites diferentes, cada uma já rotulada como positiva ou negativa. São três arquivos de texto, um por fonte, com o comentário e o rótulo separados por tabulação, no formato `comentario<TAB>classe`, sendo 0 para negativo e 1 para positivo.

Tabela 1: Composição da base consolidada

Fonte	Registros	Negativos (0)	Positivos (1)
IMDb (<code>imdb_labelled.txt</code>)	1000	500	500
Amazon (<code>amazon_cells_labelled.txt</code>)	1000	500	500
Yelp (<code>yelp_labelled.txt</code>)	1000	500	500
Total	3000	1500	1500

Cabe um comentário sobre o tamanho. O enunciado fala em 3003 registros, mas a versão atual do pacote disponibilizado pela UCI, baixada em setembro de 2026, tem exatamente mil linhas em cada arquivo, ou seja, 3000 registros no total, com as duas classes perfeitamente balanceadas. Trabalhei com esses 3000. A diferença de três linhas deve vir de alguma versão anterior do arquivo, já que a base é distribuída há bastante tempo.

3.2 Consolidação e tratamento

A junção e a limpeza foram feitas pelo script `scripts/consolidar.py`, escrito em Python. Ele lê os três arquivos, aplica os critérios de consistência e escreve o `.arff` final. Os critérios usados para descartar um registro foram os do enunciado:

- linha em branco;
- linha fora do formato, isto é, sem o separador entre o comentário e a classe;
- registro sem classificação ou com um valor de classe diferente de 0 e 1;
- registro sem comentário, com o texto vazio.

Rodando o script, nenhuma das 3000 linhas caiu nesses critérios, o que já era esperado para uma base publicada e bastante usada. Mesmo assim mantive a verificação no código, porque o objetivo do trabalho é justamente garantir a consistência e não supor que ela existe. O script também gera o arquivo `dados/registros_descartados.txt` quando alguma linha é rejeitada, para deixar registrado o que saiu e por quê.

Além dos critérios pedidos, verifiquei repetições. Comparando os textos exatamente como estão escritos, a base tem 2982 comentários distintos, o mesmo número que o Weka mostra no campo *Distinct* da Figura 1, ou seja, 18 linhas repetidas. Ignorando maiúsculas e minúsculas, são 21 linhas repetidas, vindas de 19 comentários diferentes, quase todas frases curtas como “Works great!” e “Very disappointed.”, que realmente devem ter sido escritas por pessoas diferentes. Nenhuma delas aparece com classes contraditórias, então não são inconsistência de rotulagem e resolvi mantê-las, até porque são comentários legítimos e representam bem o tipo de frase que aparece em avaliações reais. O único ajuste feito no texto foi normalizar espaços em branco repetidos, que aparecem no fim de várias frases do arquivo do IMDb e só atrapalhariam a tokenização.

3.3 A base no formato ARFF

O arquivo final, `dados/comentarios.arff`, tem a estrutura abaixo. O comentário é declarado como `string` e a classe como um atributo nominal de dois valores.

```
% Comentarios de IMDb, Amazon e Yelp (Sentiment Labelled
% Sentences - UCI Machine Learning Repository).
% 0 = comentario negativo, 1 = comentario positivo.

@relation comentarios

@attribute comentario string
@attribute classe {0,1}
```

```
@data
'A very, very, very slow-moving, aimless movie about a distressed, drifting young
man.',0
'Not sure who was more lost - the flat characters or the audience, nearly half of
whom walked out.',0
'Attempting artiness with black & white and clever camera angles, the movie
disappointed - became even more ridiculous - as the acting was poor and the
plot and lines almost non-existent.',0
'Very little music or anything to speak of.',0
'The best scene in the movie was when Gerardo is trying to find a song that keeps
running through his head.',1
```

Carregando esse arquivo no Weka, a aba *Preprocess* confirma os 3000 registros e os dois atributos (Figura 1). A Figura 2 mostra os primeiros registros pelo botão *Edit*, que é a forma mais direta de conferir se o texto e a classe ficaram nas colunas certas.

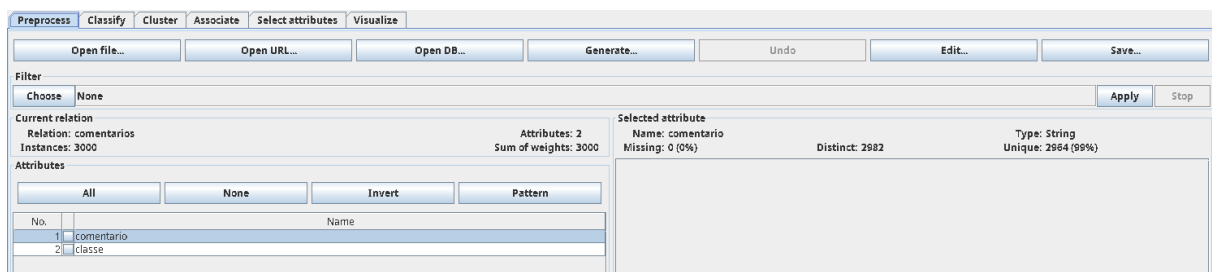


Figura 1: Base consolidada carregada no Weka: 3000 instâncias e 2 atributos.

Relation: comentarios		
No.	1: comentario String	2: classe Nominal
1	A very, very, very slow-moving, aimless movie about a distressed, drifting young man.	0
2	Not sure who was more lost - the flat characters or the audience, nearly half of whom walked out.	0
3	Attempting artiness with black & white and clever camera angles, the movie disappointed - became even more ridiculous - as the acting was poor and the pl...	0
4	Very little music or anything to speak of.	0
5	The best scene in the movie was when Gerardo is trying to find a song that keeps running through his head.	1
6	The rest of the movie lacks art, charm, meaning... If it's about emptiness, it works I guess because it's empty.	0
7	Wasted two hours.	0
8	Saw the movie today and thought it was a good effort, good messages for kids.	1
9	A bit predictable.	0
10	Loved the casting of Jimmy Buffet as the science teacher.	1
11	And those baby owls were adorable.	1
12	The movie showed a lot of Florida at it's best, made it look very appealing.	1
13	The Songs Were The Best And The Muppets Were So Hilarious.	1
14	It Was So Cool.	1
15	This is a very "right on case" movie that delivers everything almost right in your face.	1
16	It had some average acting from the main person, and it was a low budget as you clearly can see.	0
17	This review is long overdue, since I consider A Tale of Two Sisters to be the single greatest film ever made.	1
18	I'll put this gem up against any movie in terms of screenplay, cinematography, acting, post-production, editing, directing, or any other aspect of film-making.	1
19	It's practically perfect in all of them a true masterpiece in a sea of faux "masterpieces.	1
20	" The structure of this film is easily the most tightly constructed in the history of cinema.	1
21	I can think of no other film where something vitally important occurs every other minute.	1
22	In other words, the content level of this film is enough to easily fill a dozen other films.	1
23	How can anyone in their right mind ask for anything more from a movie than this?	1
24	It's quite simply the highest, most superlative form of cinema imaginable.	1
25	Yes, this film does require a rather significant amount of puzzle-solving, but the pieces fit together to create a beautiful picture.	1
26	This short film certainly pulls no punches.	0
27	Graphics is far from the best part of the game.	0
28	This is the number one best TH game in the series.	1
29	It deserves strong love.	1
30	It is an insane game.	1
31	There are massive levels, massive unlockable characters... it's just a massive game.	1

Figura 2: Primeiros registros da base de treino já configurada, vistos pelo editor do Weka.

3.4 Transformação do texto em atributos

O Naive Bayes não trabalha diretamente com um atributo do tipo `string`. É preciso transformar cada comentário em um vetor numérico, e para isso usei o filtro `StringToWordVector`, que é o filtro padrão do Weka para texto. Ele separa o comentário em palavras e cria um atributo para cada palavra do vocabulário, marcando com 1 quando a palavra aparece no comentário e 0 quando não aparece. Mantive a configuração padrão do filtro, que guarda até mil palavras por classe.

O resultado é uma base com 2092 atributos de palavras mais o atributo de classe, ainda com as mesmas 3000 instâncias, como mostra a Figura 3. Essa é a base usada em todos os testes das próximas seções, salva em `dados/comentarios_vetorizado.arff`.

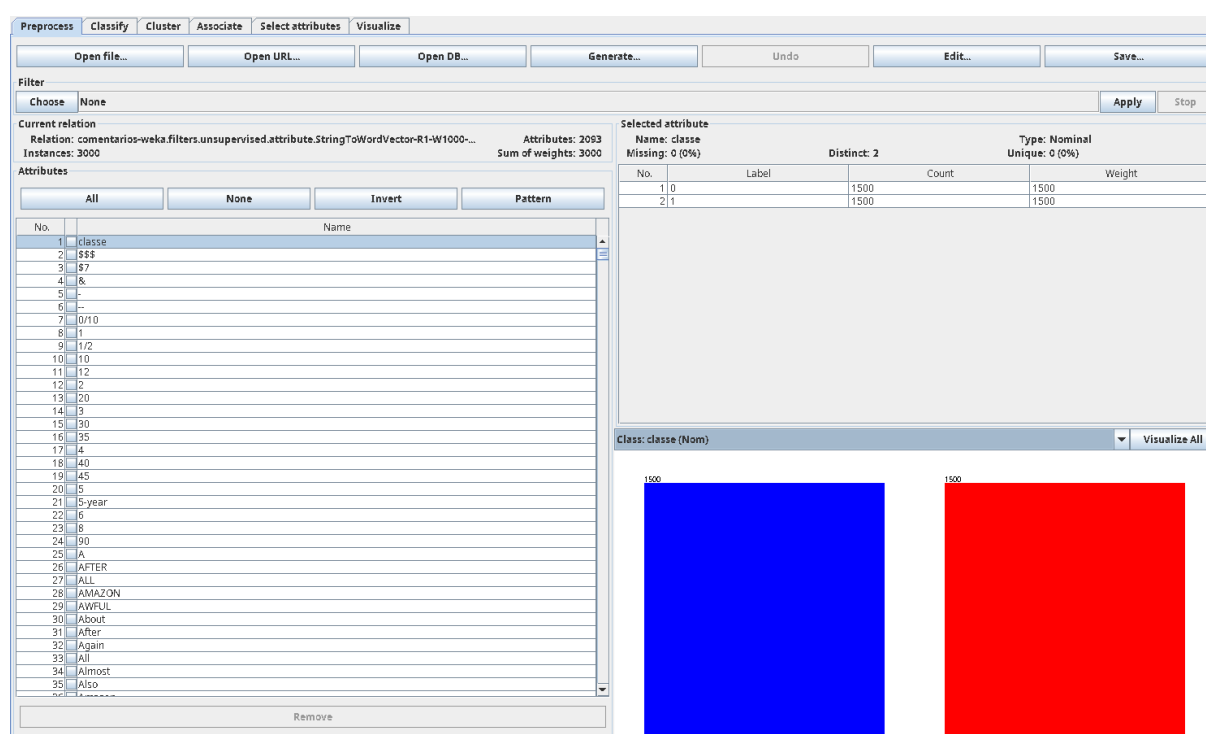


Figura 3: Base depois do *StringToWordVector*: 2093 atributos e classes balanceadas.

4 Como os testes foram feitos

Os testes foram executados no Weka 3.8.6 sobre o Java 21, em um notebook com Ubuntu. Todas as execuções usaram validação cruzada estratificada, com a semente padrão do Weka (1), o que garante que os resultados são reprodutíveis. A validação cruzada divide a base em k partes de tamanho parecido, treina o modelo k vezes usando $k - 1$ partes e testa na parte que ficou de fora, de modo que toda instância é testada exatamente uma vez por um modelo que não a viu no treino. A palavra estratificada indica que a proporção entre as classes é mantida dentro de cada parte.

Duas observações sobre como os números foram coletados. Os tempos de construção do modelo que aparecem nas tabelas vêm da execução por linha de comando, e variam um pouco de uma rodada para outra, por isso não batem na segunda casa com os valores que aparecem nas telas do Explorer. E o `StringToWordVector` foi aplicado na base inteira antes da validação cruzada, que é o caminho normal pelo Explorer, mas isso faz o vocabulário ser montado olhando também os comentários que depois entram no teste. Para medir o tamanho desse efeito, repeti a execução com o `FilteredClassifier`, que refaz o filtro dentro de cada fold: o resultado foi 69,13% contra 69,33% do procedimento usado aqui, uma diferença de 0,2 ponto percentual. Ou seja, o número não está inflado de forma relevante pelo vocabulário compartilhado.

Antes de olhar os números, vale deixar claro o que cada métrica mede, porque elas contam histórias diferentes sobre o mesmo resultado.

- **Instâncias classificadas corretamente (acurácia):** percentual de acertos sobre o total. É a métrica mais direta e funciona bem aqui porque a base é balanceada. Em bases desbalanceadas ela engana, já que um classificador que chuta sempre a classe majoritária pode ter acurácia alta sem servir para nada.
- **Taxa de verdadeiro positivo (VP ou *recall*):** dentre as instâncias que realmente pertencem a uma classe, a fração que o modelo conseguiu identificar. Para a classe 0 responde “de todos os comentários negativos, quantos eu peguei?”.
- **Taxa de falso positivo (FP):** dentre as instâncias que não pertencem à classe, a fração que o modelo classificou erradamente como sendo dela. Quanto menor, melhor. Um modelo com VP alto e FP alto ao mesmo tempo está apenas chutando aquela classe com frequência.
- **Precisão:** dentre as instâncias que o modelo apontou como sendo de uma classe, a fração que realmente era. Responde “quando eu digo que é negativo, com que frequência eu acerto?”. Precisão e recall costumam se mover em sentidos opostos.
- **F-Measure:** média harmônica entre precisão e recall, usada para resumir as duas em um número só.
- **Matriz de confusão:** tabela que mostra, para cada classe real, como as instâncias foram distribuídas entre as classes previstas. É onde dá para ver se o erro está concentrado em um lado.
- **Estatística Kappa:** compara o acerto obtido com o acerto que se conseguiria por sorte. Vale 0 quando o modelo não é melhor que o acaso e 1 quando ele acerta tudo.

- **Área ROC:** probabilidade de o modelo dar uma pontuação maior para um exemplo positivo escolhido ao acaso do que para um negativo. Diferente da acurácia, ela não depende do ponto de corte usado para decidir a classe.

5 Resultados

5.1 Naive Bayes sobre a base, com 10 folds

A primeira execução usa o Naive Bayes com as opções padrão e validação cruzada com 10 partes, que é a configuração mais comum. A saída do Weka está na Figura 4.

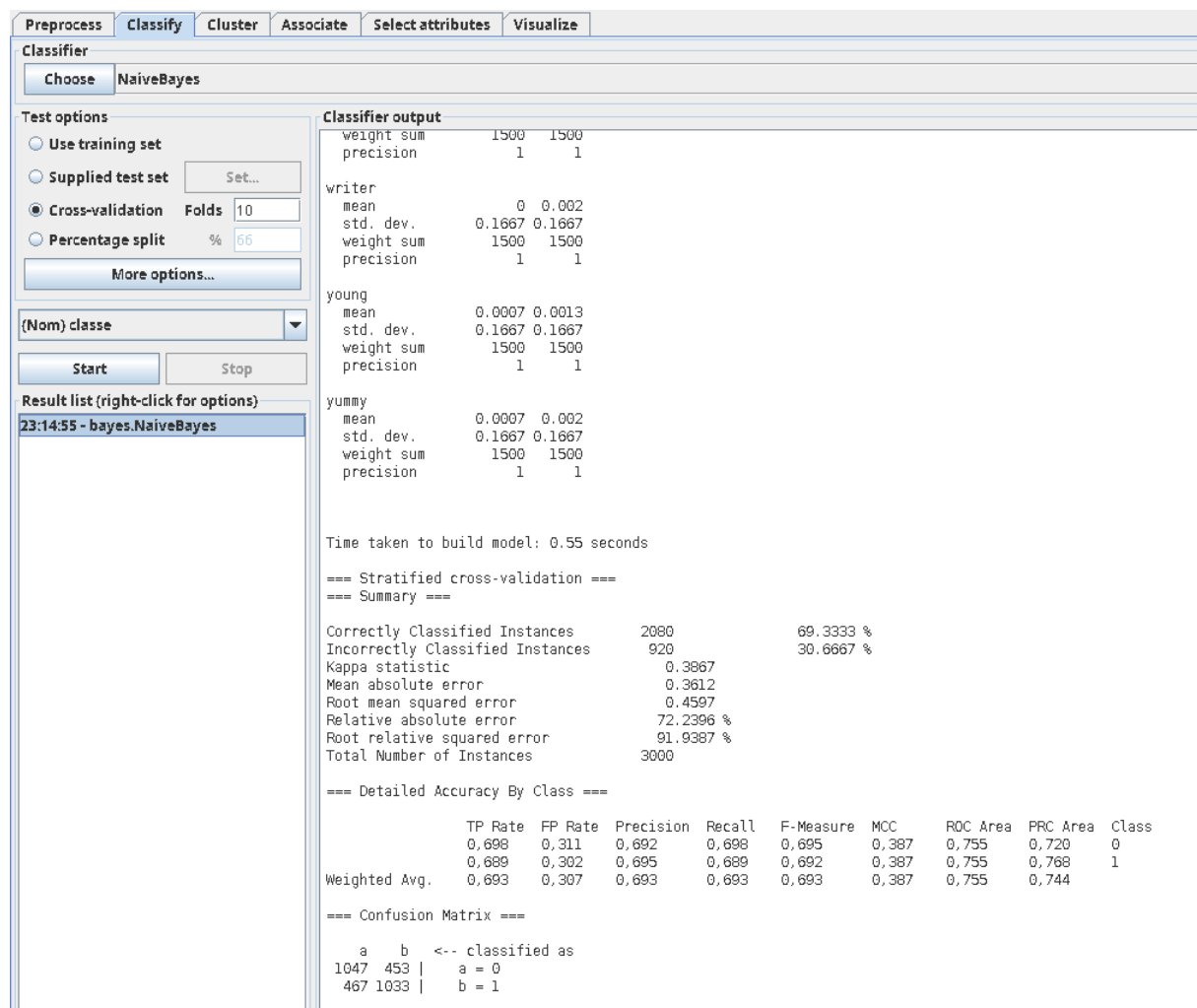


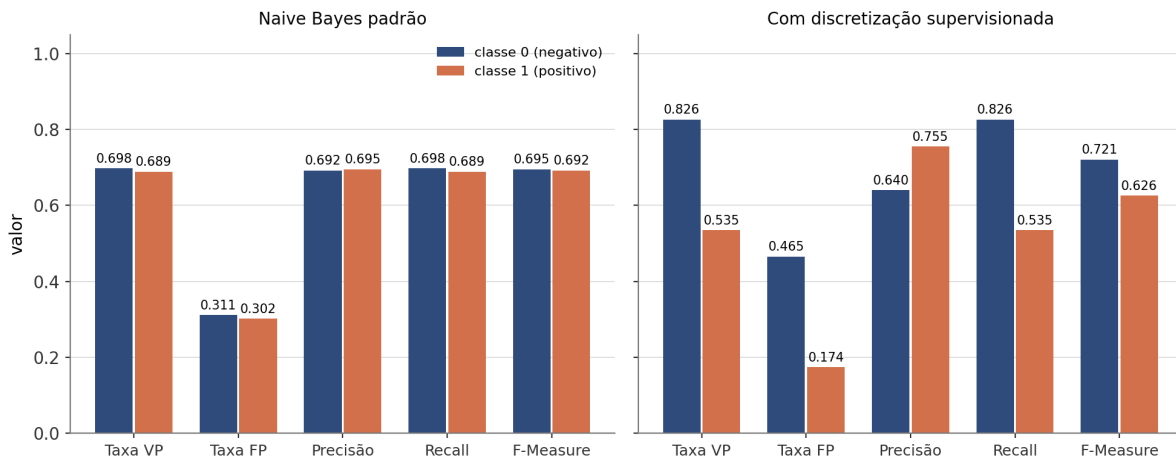
Figura 4: Saída do Naive Bayes com validação cruzada de 10 folds.

De 3000 comentários, 2080 foram classificados corretamente, o que dá 69,33% de acurácia, contra 920 erros, 30,67%. O Kappa de 0,3867 mostra que o modelo está bem acima do acaso, mas longe de um resultado forte. Como a base tem exatamente 1500 exemplos de cada classe, um classificador que chutasse sempre a mesma classe ficaria em 50%, então o ganho real do modelo é de pouco mais de 19 pontos percentuais.

Tabela 2: Métricas por classe, Naive Bayes com 10 folds

Classe	Taxa VP	Taxa FP	Precisão	Recall	F-Measure	ROC
0 (negativo)	0,698	0,311	0,692	0,698	0,695	0,755
1 (positivo)	0,689	0,302	0,695	0,689	0,692	0,755
Média ponderada	0,693	0,307	0,693	0,693	0,693	0,755

O que chama atenção na Tabela 2 é a simetria. As duas classes têm praticamente os mesmos valores em todas as métricas, o que faz sentido: a base é balanceada e nada no modelo favorece um lado. A taxa de verdadeiro positivo de 0,698 para a classe negativa significa que o modelo encontra cerca de sete de cada dez comentários negativos, enquanto a taxa de falso positivo de 0,311 diz que ele também marca como negativos cerca de três de cada dez comentários que eram positivos. A precisão de 0,692 completa o quadro: quando o modelo diz “negativo”, ele acerta em 69% das vezes. Os três números estarem na mesma faixa indica que o modelo não está enviesado para nenhuma classe, apenas erra bastante nas duas.

**Figura 5:** Métricas por classe do Naive Bayes com 10 folds, na configuração padrão e com discretização supervisionada.

A matriz de confusão dessa execução é a seguinte:

Tabela 3: Matriz de confusão, Naive Bayes com 10 folds

	Previsto 0	Previsto 1
Real 0	1047	453
Real 1	467	1033

Os erros estão distribuídos de forma quase igual entre os dois lados, 453 negativos tratados como positivos e 467 positivos tratados como negativos. Em uma aplicação real,

essa simetria é uma informação útil: o modelo não tem tendência a inflar nem a esconder a insatisfação dos clientes, ele simplesmente erra um terço das vezes.

5.2 Variando o número de folds

O enunciado pede a comparação com três valores diferentes de validação cruzada. Usei 5, 10 e 20 folds. O resumo está na Tabela 4 e na Figura 6. As saídas completas das execuções de 5 e 20 folds estão no Apêndice B.

Tabela 4: Naive Bayes com diferentes valores de validação cruzada

Folds	Corretas	Incorretas	Acurácia	VP	FP	Precisão	Kappa
5	2075	925	69,17%	0,692	0,308	0,692	0,3833
10	2080	920	69,33%	0,693	0,307	0,693	0,3867
20	2106	894	70,20%	0,702	0,298	0,702	0,4040

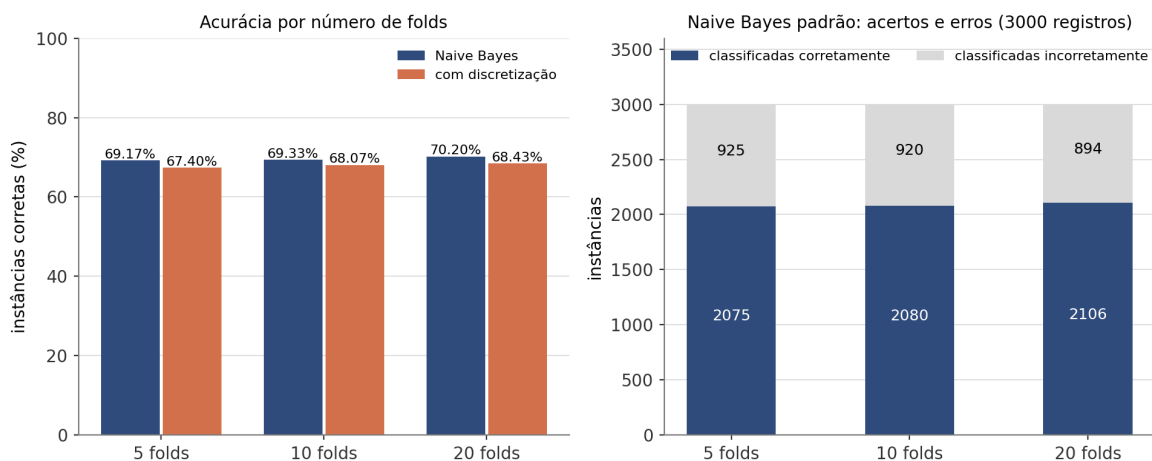


Figura 6: Acurácia por número de folds e distribuição de acertos e erros.

A diferença entre as três configurações é pequena, pouco mais de um ponto percentual entre o pior e o melhor caso, e cresce junto com o número de folds. O motivo é direto: com 5 folds, cada modelo é treinado com 2400 exemplos; com 20 folds, com 2850. Mais dados de treino em cada rodada produzem um modelo um pouco melhor, e como cada teste é feito sobre uma fatia menor, o resultado final acaba sendo a média de vinte estimativas em vez de cinco. O preço é o tempo, já que com 20 folds o Weka precisa treinar vinte modelos.

As matrizes de confusão acompanham esse movimento (Figura 7). Com 5 folds são 450 negativos classificados como positivos e 475 positivos classificados como negativos; com 20 folds esses números caem para 429 e 465. A melhora vem dos dois lados, sem deslocar o equilíbrio entre as classes.

Tabela 5: Matrizes de confusão por número de folds (linha = classe real)

Folds	Naive Bayes		Com discretização	
	Prev. 0	Prev. 1	Prev. 0	Prev. 1
5 (real 0)	1050	450	1229	271
5 (real 1)	475	1025	707	793
10 (real 0)	1047	453	1239	261
10 (real 1)	467	1033	697	803
20 (real 0)	1071	429	1249	251
20 (real 1)	465	1035	696	804

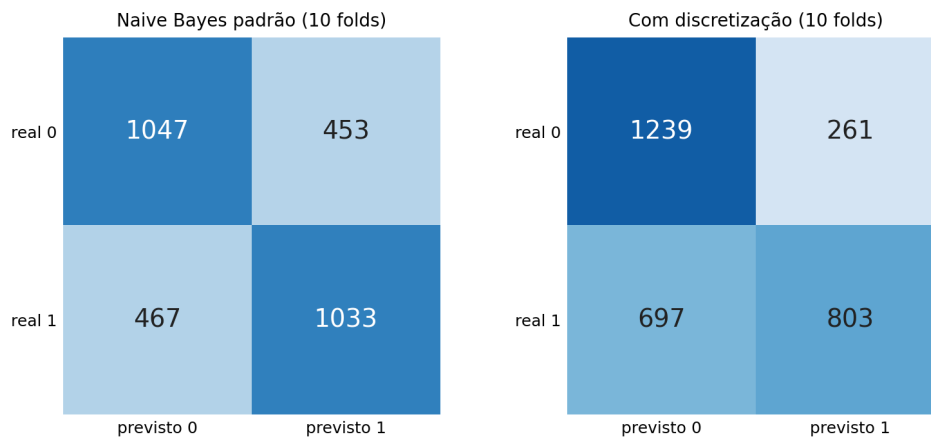


Figura 7: Matrizes de confusão com 10 folds, nas duas configurações do Naive Bayes.

Vale dizer que uma variação de um ponto percentual, com 3000 instâncias, está dentro da faixa que se espera de flutuação entre partições diferentes. Não dá para afirmar que 20 folds é sempre melhor; o que dá para afirmar é que o classificador é estável, e isso é bom sinal, porque mostra que o resultado de 69% a 70% não depende da forma como a base foi dividida.

5.3 Ligando a discretização supervisionada

A opção *useSupervisedDiscretization* fica na janela de configuração do classificador, aberta ao clicar na caixa de texto ao lado do botão *Choose* (Figura 8). Com ela ligada, o Weka deixa de tratar cada atributo como uma variável real com distribuição normal e passa a discretizá-lo antes, usando o método de Fayyad e Irani, que procura pontos de corte com base no ganho de informação em relação à classe. Depois disso as probabilidades são estimadas por contagem, como acontece com atributos nominais.

A taxa de acerto com a opção ligada e 10 folds foi de **68,07%**, ou 2042 instâncias corretas, contra 69,33% da configuração padrão. Rodando também com 5 e 20 folds o comportamento se repete, com 67,40% e 68,43%.

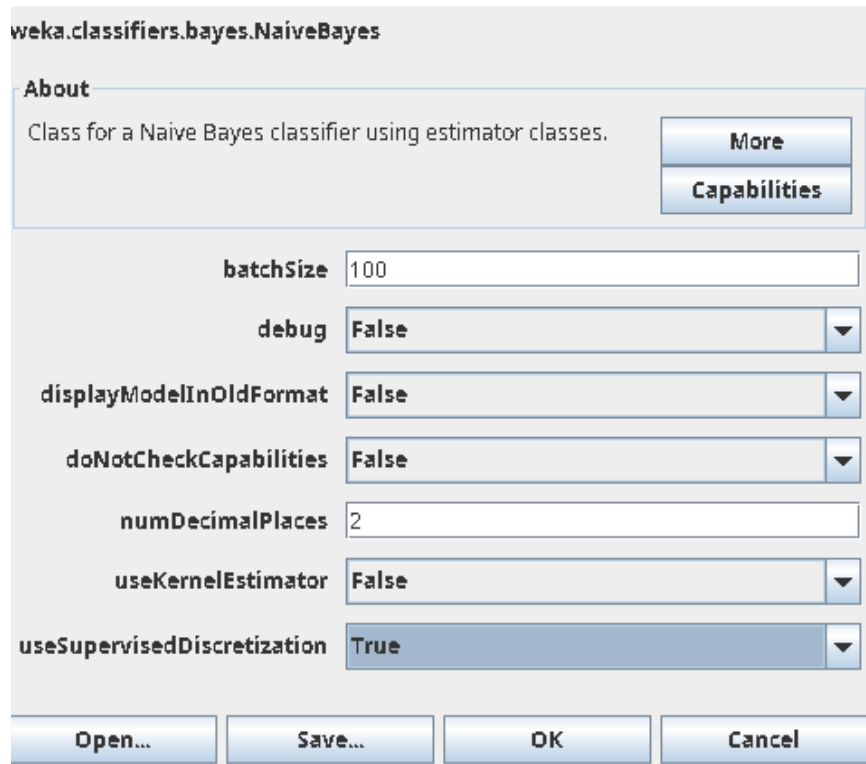


Figura 8: Janela de opções do Naive Bayes com *useSupervisedDiscretization* em *True*.

Tabela 6: Naive Bayes padrão e com discretização supervisionada

Configuração	Acurácia	VP	FP	Precisão	Kappa	ROC
Padrão, 5 folds	69,17%	0,692	0,308	0,692	0,3833	0,753
Padrão, 10 folds	69,33%	0,693	0,307	0,693	0,3867	0,755
Padrão, 20 folds	70,20%	0,702	0,298	0,702	0,4040	0,758
Discretizado, 5 folds	67,40%	0,674	0,326	0,690	0,3480	0,754
Discretizado, 10 folds	68,07%	0,681	0,319	0,697	0,3613	0,761
Discretizado, 20 folds	68,43%	0,684	0,316	0,702	0,3687	0,764

A mudança foi significativa? Em acurácia, não. A diferença é de 1,27 ponto percentual para baixo com 10 folds, o que para 3000 instâncias representa 38 comentários a mais classificados errado. Isso está próximo da variação que já aparece só por mudar o número de folds, então seria exagero chamar de queda expressiva. Em compensação, o comportamento do classificador mudou bastante, e é aí que está o efeito interessante. Comparando as duas colunas da Figura 5, a simetria entre as classes desaparece. A taxa de verdadeiro positivo da classe negativa sobe de 0,698 para 0,826, mas a da classe positiva desaba de 0,689 para 0,535. A taxa de falso positivo da classe negativa vai de 0,311 para 0,465. O modelo passou a dizer “negativo” com muito mais frequência: das 3000 previsões, 1936 foram para a classe 0 e apenas 1064 para a classe 1, quando o correto seria 1500 para cada lado.

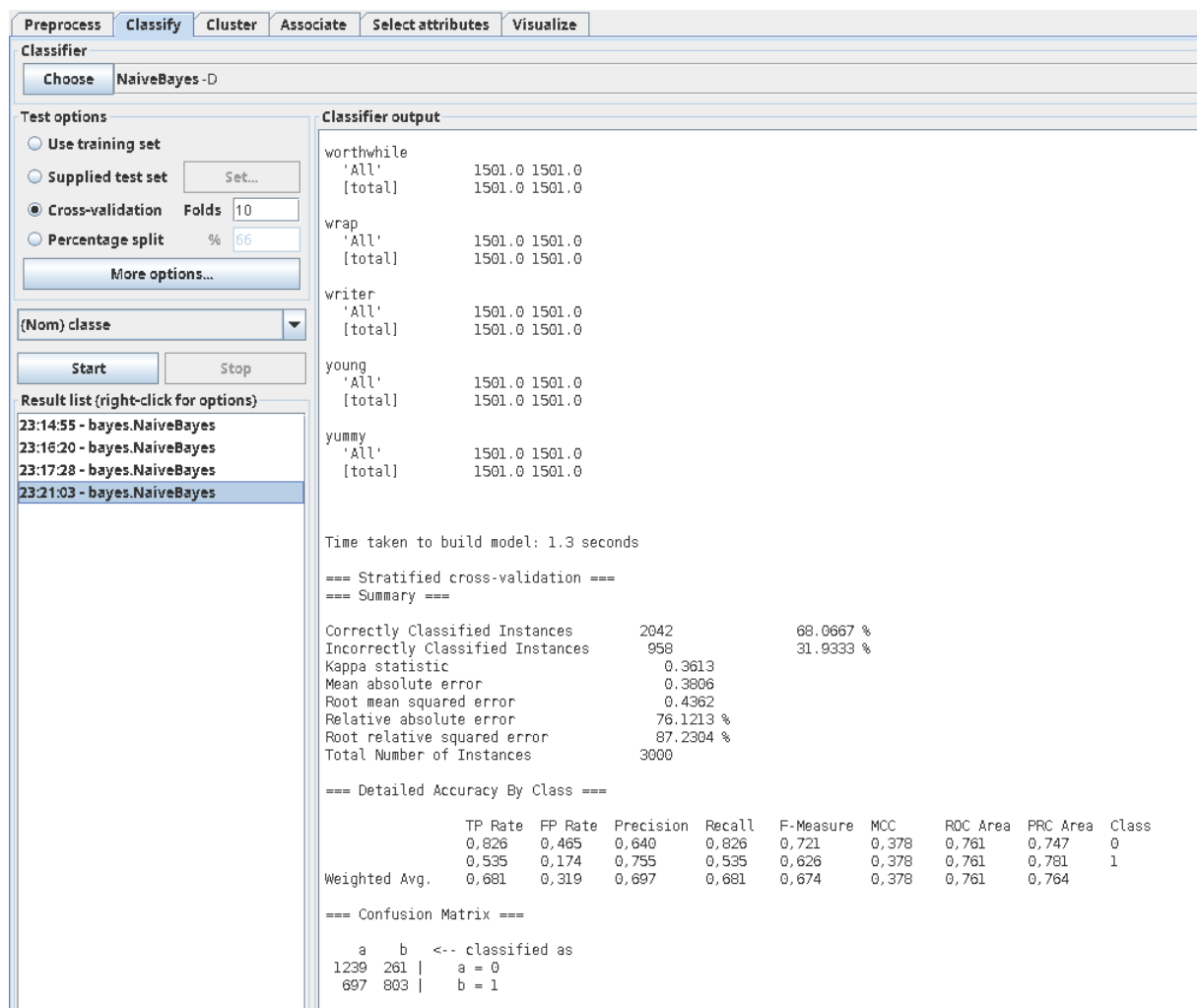


Figura 9: Resultado com discretização supervisionada, 10 folds.

Por que isso aconteceu? A resposta está na natureza dos atributos criados pelo `StringToWordVector`. Cada um deles só assume dois valores, 0 se a palavra não aparece no comentário e 1 se aparece, e a esmagadora maioria das palavras é rara, aparecendo em pouquíssimos comentários. O método de Fayyad e Irani só cria um ponto de corte quando o ganho de informação compensa o custo de descrição do corte, pelo critério MDL. Para uma palavra que aparece em vinte comentários de três mil, esse ganho não compensa, e o atributo inteiro é colapsado em um único intervalo, ou seja, deixa de distinguir qualquer coisa.

Para confirmar isso, apliquei o filtro `supervised.attribute.Discretize` sobre a base vetorizada e contei o resultado: dos 2092 atributos de palavras, 2063 viraram um intervalo único e apenas 29 receberam um corte de verdade. As 29 palavras que sobreviveram são justamente as mais carregadas de sentimento:

amazing, and, bad, don, good, great, horrible, love, minutes, money, nice, not, poor, slow, stupid, t, terrible, waste, works, worst, Good, Great, awesome, beautiful, delicious, excellent, fine, loved, wonderful.

Isso explica os dois efeitos ao mesmo tempo. A acurácia cai porque o modelo joga fora a contribuição pequena, mas somada relevante, de mais de dois mil atributos que na versão gaussiana ainda empurravam um pouco a probabilidade para um lado ou para o outro. E o viés aparece porque as palavras que sobraram não são equilibradas: o vocabulário negativo forte (“bad”, “worst”, “waste”, “horrible”, “poor”) é mais concentrado e mais repetido que o positivo, além de “not” e “don” entrarem quase sempre em contexto de crítica. Um comentário positivo que não usa nenhuma das palavras marcantes fica sem evidência a favor da classe 1 e acaba caindo na classe 0.

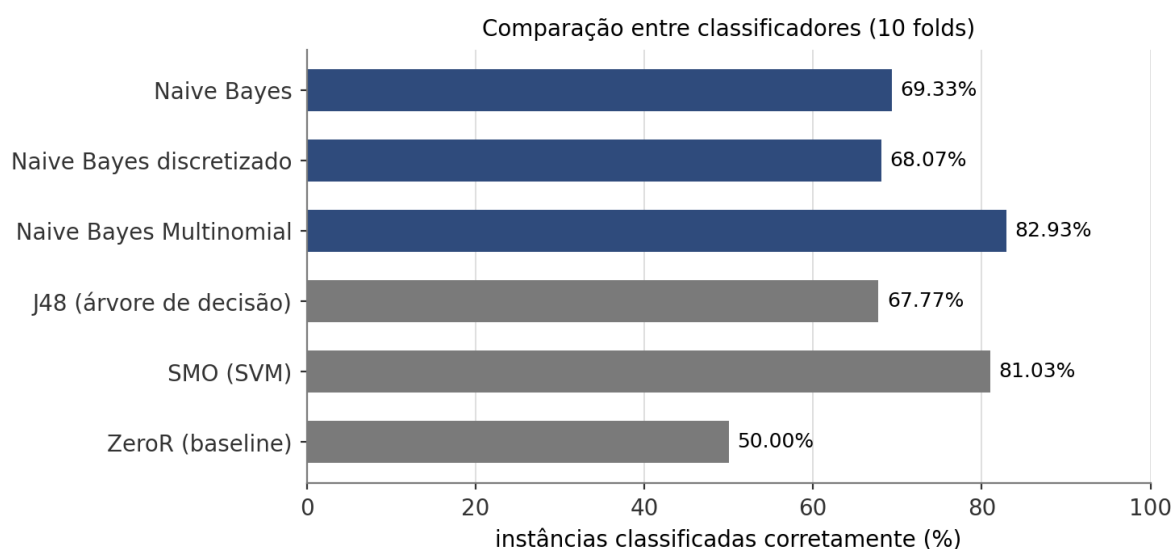
Um detalhe que reforça a leitura: a área ROC subiu um pouco com a discretização, de 0,755 para 0,761. Ou seja, a ordenação das probabilidades ficou até um pouco melhor, e o que piorou foi a decisão no corte de 0,5, por causa do desequilíbrio que o modelo passou a ter. Se o problema exigisse encontrar o máximo possível de comentários negativos, por exemplo em um sistema de alerta de reclamações, a versão discretizada seria a preferível apesar da acurácia menor, porque captura 82,6% dos negativos contra 69,8% da versão padrão.

5.4 Comparação com outros classificadores

Para ter uma referência do que esses 69% significam, rodei mais quatro classificadores do Weka sobre exatamente a mesma base e com 10 folds. Essa parte não era obrigatória, mas ajuda a responder a última pergunta do enunciado.

Tabela 7: Comparação entre classificadores, 10 folds

Classificador	Acurácia	VP	FP	Kappa	Tempo
Naive Bayes Multinomial	82,93%	0,829	0,171	0,6587	0,05 s
SMO (SVM linear)	81,03%	0,810	0,190	0,6207	6,10 s
Naive Bayes	69,33%	0,693	0,307	0,3867	0,57 s
Naive Bayes discretizado	68,07%	0,681	0,319	0,3613	1,01 s
J48 (árvore de decisão)	67,77%	0,678	0,322	0,3553	13,95 s
ZeroR (referência)	50,00%	0,500	0,500	0,0000	0,04 s

**Figura 10:** Instâncias classificadas corretamente por classificador, 10 folds.

O resultado mais interessante é o do Naive Bayes Multinomial, que chega a 82,93% com o menor tempo de treino de todos. Ele usa o mesmo teorema de Bayes e a mesma suposição de independência, mudando apenas a forma de estimar as probabilidades: em vez de ajustar uma normal a cada atributo, ele modela o comentário como uma sequência de palavras sorteadas de uma distribuição por classe. Para texto, essa é a modelagem correta, e a diferença de 13 pontos percentuais mostra o tamanho do problema que existe em tratar um atributo binário de presença de palavra como se fosse uma variável contínua com distribuição normal. O SMO, que é a implementação de máquina de vetores de suporte do Weka, fica logo atrás com 81,03%, mas leva mais de cem vezes o tempo do multinomial. O J48 é o mais lento e um dos piores aqui, o que faz sentido: uma árvore precisa escolher poucos atributos para dividir a base, e em texto a informação está espalhada por muitas palavras ao mesmo tempo.

6 Resumo dos resultados

Juntando tudo, a Tabela 7 responde às duas perguntas finais do enunciado, separando performance de qualidade de classificação.

Em **performance**, no sentido de custo de treino, o vencedor é o Naive Bayes Multinomial, com 0,05 segundo para construir o modelo, seguido pelo Naive Bayes padrão, com 0,57 segundo. Dentro das variações pedidas, o Naive Bayes padrão é quase duas vezes mais rápido que a versão com discretização supervisionada, já que esta precisa analisar os pontos de corte de todos os 2092 atributos antes de contar as frequências. O J48 gasta quase 14 segundos, cerca de 25 vezes mais que o Naive Bayes, para entregar um resultado pior.

Em **resultado de classificação**, o melhor é de novo o Naive Bayes Multinomial, com 82,93% de acerto, Kappa de 0,6587 e taxa de falso positivo de 0,171. Entre as duas configurações pedidas no trabalho, o Naive Bayes padrão classifica melhor que a versão discretizada, 69,33% contra 68,07% com 10 folds, e mantém o equilíbrio entre as classes, que a versão discretizada perde.

Uma observação final sobre a ligação entre as duas coisas. Tanto o melhor resultado quanto o melhor tempo ficaram com o mesmo algoritmo, o que não é o caso mais comum. Em geral se paga tempo por qualidade, como acontece com o SMO, que fica a dois pontos do multinomial gastando 122 vezes mais tempo. O que o experimento mostra é que, para classificação de texto, escolher a modelagem certa das probabilidades importa mais que escolher um algoritmo mais caro.

7 Conclusão

O trabalho consolidou as três fontes de comentários em uma base única de 3000 registros, verificada quanto a formato, rótulo e conteúdo, e convertida para ARFF. Sobre essa base, o Naive Bayes com as opções padrão do Weka classificou corretamente 69,33% dos comentários com 10 folds, com métricas praticamente iguais para as duas classes e Kappa de 0,3867. Variar a validação cruzada entre 5, 10 e 20 folds moveu o resultado em pouco mais de um ponto percentual, o que indica estabilidade e não dependência da partição.

Ligar a discretização supervisionada não trouxe ganho de acerto, ficando 1,27 ponto abaixo da configuração padrão, e o motivo pôde ser verificado diretamente: com atributos binários de presença de palavra, o critério MDL descarta 2063 dos 2092 atributos, e o modelo passa a decidir com base em 29 palavras, ficando mais sensível aos comentários negativos e menos aos positivos.

O ponto que eu levaria desse experimento é que o limite aqui não está no Naive Bayes, e sim em como o texto foi representado e em qual estimador de probabilidade

combina com essa representação. O mesmo algoritmo, com a variante multinomial, saltou de 69% para 83% sobre exatamente os mesmos dados e no menor tempo de treino de todos os testes.

Referências

- KOTZIAS, D.; DENIL, M.; DE FREITAS, N.; SMYTH, P. *From Group to Individual Labels using Deep Features*. KDD, 2015. *Base Sentiment Labelled Sentences*, UCI Machine Learning Repository. Disponível em: <https://archive.ics.uci.edu/ml/datasets/Sentiment+Labelled+Sentences>.
- WITTEN, I. H.; FRANK, E.; HALL, M. A.; PAL, C. J. *Data Mining: Practical Machine Learning Tools and Techniques*. 4. ed. Morgan Kaufmann, 2016.
- FAYYAD, U. M.; IRANI, K. B. *Multi-interval discretization of continuous-valued attributes for classification learning*. IJCAI, 1993.
- MCCALLUM, A.; NIGAM, K. *A comparison of event models for Naive Bayes text classification*. AAAI Workshop on Learning for Text Categorization, 1998.
- Documentação do Weka 3.8. Disponível em: <https://www.cs.waikato.ac.nz/ml/weka/>.

A Como reproduzir os resultados

Os arquivos do trabalho estão organizados assim:

```
dados/      base original, comentarios.arff e comentarios_vetorizado.arff
scripts/    consolidar.py, rodar_weka.sh e gerar_graficos.py
resultados/ saidas completas do Weka em txt e o resumo em csv
imagens/    telas do Weka e graficos usados no relatorio
relatorio/  este documento
```

Para refazer tudo do zero:

```
python3 scripts/consolidar.py
WEKA_CP=/caminho/weka.jar ./scripts/rodar_weka.sh
python3 scripts/gerar_graficos.py
```

Pelo Weka Explorer, o caminho é abrir `dados/comentarios.arff`, aplicar o filtro `unsupervised.attribute.StringToWordVector`, ir para a aba *Classify*, selecionar `bayes.NaiveBayes`, conferir que o atributo de classe é `classe`, escolher o número de folds e clicar em *Start*.

B Saídas do Weka com 5 e 20 folds

Classifier

Choose NaiveBayes

Test options

☐ Use training set

☐ Supplied test set Set...

☒ Cross-validation Folds 5

☐ Percentage split % 66

More options...

{Nom} classe

Start Stop

Result list (right-click for options)

23:14:55 - bayes.NaiveBayes

23:16:20 - bayes.NaiveBayes

Classifier output

```

weight sum      1500  1500
precision        1      1

writer
mean              0  0.002
std. dev.        0.1667 0.1667
weight sum      1500  1500
precision        1      1

young
mean              0.0007 0.0013
std. dev.        0.1667 0.1667
weight sum      1500  1500
precision        1      1

yummy
mean              0.0007 0.002
std. dev.        0.1667 0.1667
weight sum      1500  1500
precision        1      1

Time taken to build model: 0.43 seconds

=== Stratified cross-validation ===
=== Summary ===

Correctly Classified Instances      2075      69.1667 %
Incorrectly Classified Instances    925      30.8333 %
Kappa statistic                    0.3833
Mean absolute error                 0.3616
Root mean squared error             0.4612
Relative absolute error             72.3157 %
Root relative squared error         92.2439 %
Total Number of Instances          3000

=== Detailed Accuracy By Class ===
               TP Rate  FP Rate  Precision  Recall   F-Measure  MCC      ROC Area  PRC Area  Class
               0.700    0.317    0.689     0.700    0.694     0.383    0.753    0.719     0
               0.683    0.300    0.695     0.683    0.689     0.383    0.753    0.764     1
Weighted Avg.   0.692    0.308    0.692     0.692    0.692     0.383    0.753    0.741

=== Confusion Matrix ===
      a    b  <-- classified as
1050  450 |    a = 0
 475 1025 |    b = 1
  
```

Figura 11: Naive Bayes com validação cruzada de 5 folds.

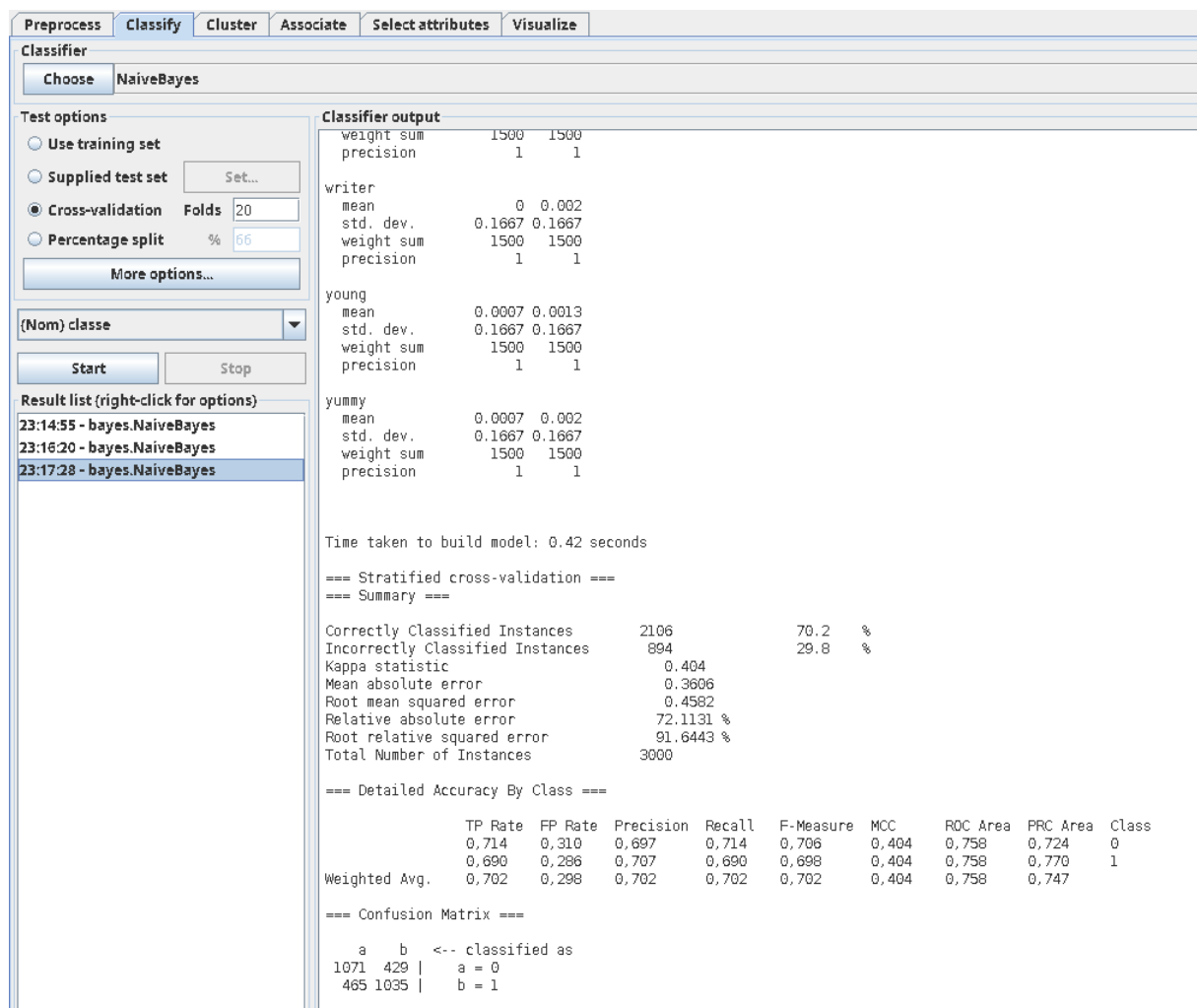


Figura 12: Naive Bayes com validação cruzada de 20 folds.